Fully connected networks

In the previous notebook, you implemented a simple two-layer neural network class. However, this class is not modular. If you wanted to change the number of layers, you would need to write a new loss and gradient function. If you wanted to optimize the network with different optimizers, you'd need to write new training functions. If you wanted to incorporate regularizations, you'd have to modify the loss and gradient function.

Instead of having to modify functions each time, for the rest of the class, we'll work in a more modular framework where we define forward and backward layers that calculate losses and gradients respectively. Since the forward and backward layers share intermediate values that are useful for calculating both the loss and the gradient, we'll also have these function return "caches" which store useful intermediate values.

The goal is that through this modular design, we can build different sized neural networks for various applications.

In this HW #3, we'll define the basic architecture, and in HW #4, we'll build on this framework to implement different optimizers and regularizations (like BatchNorm and Dropout).

CS231n has built a solid API for building these modular frameworks and training them, and we will use their very well implemented framework as opposed to "reinventing the wheel." This includes using their Solver, various utility functions, and their layer structure. This also includes nndl.fc_net, nndl.layers, and nndl.layer_utils. As in prior assignments, we thank Serena Yeung & Justin Johnson for permission to use code written for the CS 231n class (cs231n.stanford.edu).

Modular layers

This notebook will build modular layers in the following manner. First, there will be a forward pass for a given layer with inputs (`x`) and return the output of that layer (`out`) as well as cached variables (`cache`) that will be used to calculate the gradient in the backward pass.

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
  out = # the output

  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive derivative of loss with respect to outputs and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

In [1]:
```
## Import and setups

import time
import numpy as np
import matplotlib.pyplot as plt
from nndl.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

In [2]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k in data.keys():
  print('{}: {} '.format(k, data[k].shape))
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

Linear layers

In this section, we'll implement the forward and backward pass for the linear layers.

The linear layer forward pass is the function `affine_forward` in `nndl/layers.py` and the backward pass is `affine_backward`.

After you have implemented these, test your implementation by running the cell below.

Affine layer forward pass

Implement `affine_forward` and then test your code by running the following cell.

In [3]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_sha
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around 1e-9.
print('Testing affine_forward function:')
print('difference: {}'.format(rel_error(out, correct_out)))
```

```
Testing affine_forward function:
difference: 9.769849468192957e-10
```

Affine layer backward pass

Implement `affine_backward` and then test your code by running the following cell.

In [4]:
```
# Test the affine_backward function

x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around 1e-10
print('Testing affine_backward function:')
print('dx error: {}'.format(rel_error(dx_num, dx)))
```

```
print('dw error: {}'.format(rel_error(dw_num, dw)))
print('db error: {}'.format(rel_error(db_num, db)))
```

```
Testing affine_backward function:
dx error: 7.907681593835214e-10
dw error: 3.3066359355528596e-10
db error: 6.795685876004848e-11
```

Activation layers

In this section you'll implement the ReLU activation.

ReLU forward pass

Implement the `relu_forward` function in `nndl/layers.py` and then test your code by running the following cell.

In [5]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,
                        [ 0.,          0.,          0.04545455,  0.136363
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,

# Compare your output with ours. The error should be around 1e-8
print('Testing relu_forward function:')
print('difference: {}'.format(rel_error(out, correct_out)))
```

```
Testing relu_forward function:
difference: 4.999999798022158e-08
```

ReLU backward pass

Implement the `relu_backward` function in `nndl/layers.py` and then test your code by running the following cell.

In [6]:
```
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, d

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be around 1e-12
print('Testing relu_backward function:')
print('dx error: {}'.format(rel_error(dx_num, dx)))
```

```
Testing relu_backward function:
dx error: 3.2756068888715097e-12
```

Combining the affine and ReLU layers

Often times, an affine layer will be followed by a ReLU layer. So let's make one that puts them together. Layers that are combined are stored in `nndl/layer_utils.py`.

Affine-ReLU layers

We've implemented `affine_relu_forward()` and `affine_relu_backward` in `nndl/layer_utils.py`. Take a look at them to make sure you understand what's going on. Then run the following cell to ensure its implemented correctly.

In [7]:
```
from nndl.layer_utils import affine_relu_forward, affine_relu_backward

x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w

print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: {}'.format(rel_error(dx_num, dx)))
print('dw error: {}'.format(rel_error(dw_num, dw)))
print('db error: {}'.format(rel_error(db_num, db)))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error: 7.670477715174643e-11
dw error: 3.575189140654874e-08
db error: 5.198111178731233e-12
```

Softmax and SVM losses

You've already implemented these, so we have written these in `layers.py`. The following code will ensure they are working correctly.

In [8]:
```
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: svm_loss(x, y)[0], x, verbose=
loss, dx = svm_loss(x, y)

# Test svm_loss function. Loss should be around 9 and dx error should be
print('Testing svm_loss:')
print('loss: {}'.format(loss))
print('dx error: {}'.format(rel_error(dx_num, dx)))

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x, verb
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be 2.3 and dx error should be 1
```

```
print('\nTesting softmax_loss:')
print('loss: {}'.format(loss))
print('dx error: {}'.format(rel_error(dx_num, dx)))
```

```
Testing svm_loss:
loss: 8.999012479319633
dx error: 3.038735505103329e-09

Testing softmax_loss:
loss: 2.302486809092634
dx error: 7.829228850343858e-09

Testing softmax_loss:
loss: 2.302486809092634
dx error: 7.829228850343858e-09
```

Implementation of a two-layer NN

In `nndl/fc_net.py`, implement the class `TwoLayerNet` which uses the layers you made here. When you have finished, the following cell will test your implementation.

In [9]:
```
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-2
model = TwoLayerNet(input_dim=D, hidden_dims=H, num_classes=C, weight_sca

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434,
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128,
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822,
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss
```

```
model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization los

for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = {}'.format(reg))
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=Fal
    print('{} relative error: {}'.format(name, rel_error(grad_num, grads[
```

```
Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.5215703686475096e-08
W2 relative error: 3.2068321167375225e-10
b1 relative error: 8.368195737354163e-09
b2 relative error: 4.3291360264321544e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 2.527915175868136e-07
W2 relative error: 2.8508510893102143e-08
b1 relative error: 1.5646801536371197e-08
b2 relative error: 7.759095355706557e-10
```

Solver

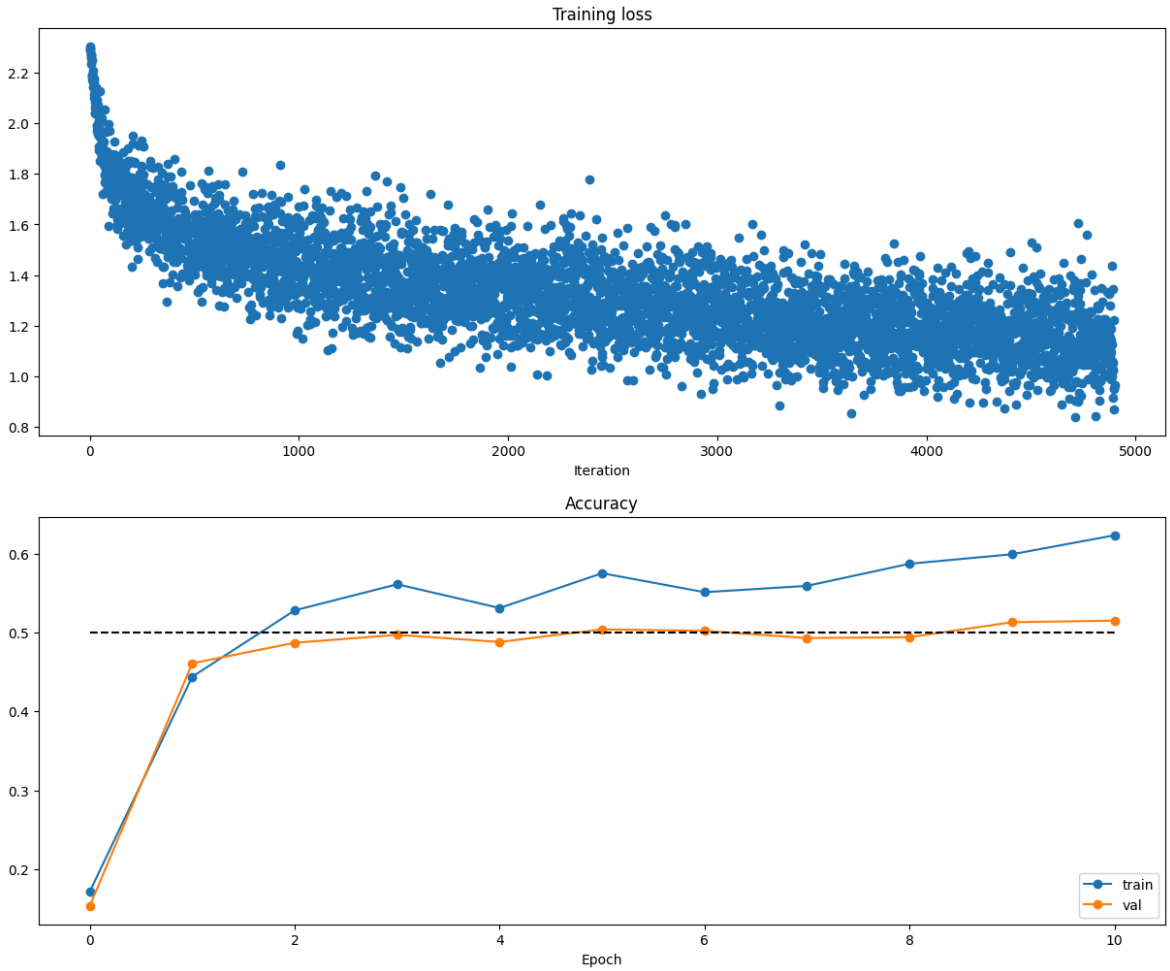
We will now use the cs231n Solver class to train these networks. Familiarize yourself with the API in `cs231n/solver.py`. After you have done so, declare an instance of a TwoLayerNet with 200 units and then train it with the Solver. Choose parameters so that your validation accuracy is at least 40%.

In [10]:
```
# model = TwoLayerNet()
# solver = None

# ================================================================ #
# YOUR CODE HERE:
#   Declare an instance of a TwoLayerNet and then train
#   it with the Solver. Choose hyperparameters so that your validation
#   accuracy is at least 40%.  We won't have you optimize this further
#   since you did it in the previous notebook.
#
# ================================================================ #

model = TwoLayerNet(hidden_dims=100, reg=1e-3)
solver = Solver(model, data,
                update_rule='sgd',
                optim_config={
                  'learning_rate': 1e-3,
                },
                lr_decay=0.95,
                num_epochs=10, batch_size=100,
                print_every=100)
```

```
solver.train()

# ================================================================ #
# END YOUR CODE HERE
# ================================================================ #
```

```
(Iteration 1 / 4900) loss: 2.305185
(Epoch 0 / 10) train acc: 0.172000; val_acc: 0.154000
(Iteration 101 / 4900) loss: 1.776363
(Iteration 201 / 4900) loss: 1.596689
(Iteration 301 / 4900) loss: 1.605871
(Iteration 401 / 4900) loss: 1.506196
(Epoch 1 / 10) train acc: 0.444000; val_acc: 0.461000
(Iteration 501 / 4900) loss: 1.400404
(Iteration 601 / 4900) loss: 1.448401
(Iteration 701 / 4900) loss: 1.309767
(Iteration 801 / 4900) loss: 1.451530
(Iteration 901 / 4900) loss: 1.272981
(Epoch 2 / 10) train acc: 0.528000; val_acc: 0.487000
(Iteration 1001 / 4900) loss: 1.331897
(Iteration 1101 / 4900) loss: 1.313353
(Iteration 1201 / 4900) loss: 1.510014
(Iteration 1301 / 4900) loss: 1.627897
(Iteration 1401 / 4900) loss: 1.290952
(Epoch 3 / 10) train acc: 0.561000; val_acc: 0.497000
(Iteration 1501 / 4900) loss: 1.274570
(Iteration 1601 / 4900) loss: 1.283218
(Iteration 1701 / 4900) loss: 1.399117
(Iteration 1801 / 4900) loss: 1.302109
(Iteration 1901 / 4900) loss: 1.659826
(Epoch 4 / 10) train acc: 0.531000; val_acc: 0.488000
(Iteration 2001 / 4900) loss: 1.405826
(Iteration 2101 / 4900) loss: 1.272642
(Iteration 2201 / 4900) loss: 1.299231
(Iteration 2301 / 4900) loss: 1.082287
(Iteration 2401 / 4900) loss: 1.027772
(Epoch 5 / 10) train acc: 0.575000; val_acc: 0.504000
(Iteration 2501 / 4900) loss: 1.399512
(Iteration 2601 / 4900) loss: 1.148213
(Iteration 2701 / 4900) loss: 1.346309
(Iteration 2801 / 4900) loss: 1.279500
(Iteration 2901 / 4900) loss: 1.162905
(Epoch 6 / 10) train acc: 0.551000; val_acc: 0.502000
(Iteration 3001 / 4900) loss: 1.288684
(Iteration 3101 / 4900) loss: 1.163541
(Iteration 3201 / 4900) loss: 1.263249
(Iteration 3301 / 4900) loss: 1.077180
(Iteration 3401 / 4900) loss: 1.191017
(Epoch 7 / 10) train acc: 0.559000; val_acc: 0.493000
(Iteration 3501 / 4900) loss: 1.100434
(Iteration 3601 / 4900) loss: 1.153963
(Iteration 3701 / 4900) loss: 1.153922
(Iteration 3801 / 4900) loss: 1.182545
(Iteration 3901 / 4900) loss: 1.068230
(Epoch 8 / 10) train acc: 0.587000; val_acc: 0.494000
(Iteration 4001 / 4900) loss: 1.029567
(Iteration 4101 / 4900) loss: 1.119321
(Iteration 4201 / 4900) loss: 1.169232
(Iteration 4301 / 4900) loss: 1.260887
(Iteration 4401 / 4900) loss: 1.491188
(Epoch 9 / 10) train acc: 0.599000; val_acc: 0.513000
(Iteration 4501 / 4900) loss: 1.045710
(Iteration 4601 / 4900) loss: 1.170750
(Iteration 4701 / 4900) loss: 1.053780
(Iteration 4801 / 4900) loss: 1.191564
(Epoch 10 / 10) train acc: 0.623000; val_acc: 0.515000
```

In [11]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```



Multilayer Neural Network

Now, we implement a multi-layer neural network.

Read through the `FullyConnectedNet` class in the file `nndl/fc_net.py`.

Implement the initialization, the forward pass, and the backward pass. There will be lines for batchnorm and dropout layers and caches; ignore these all for now. That'll be in assignment #4.

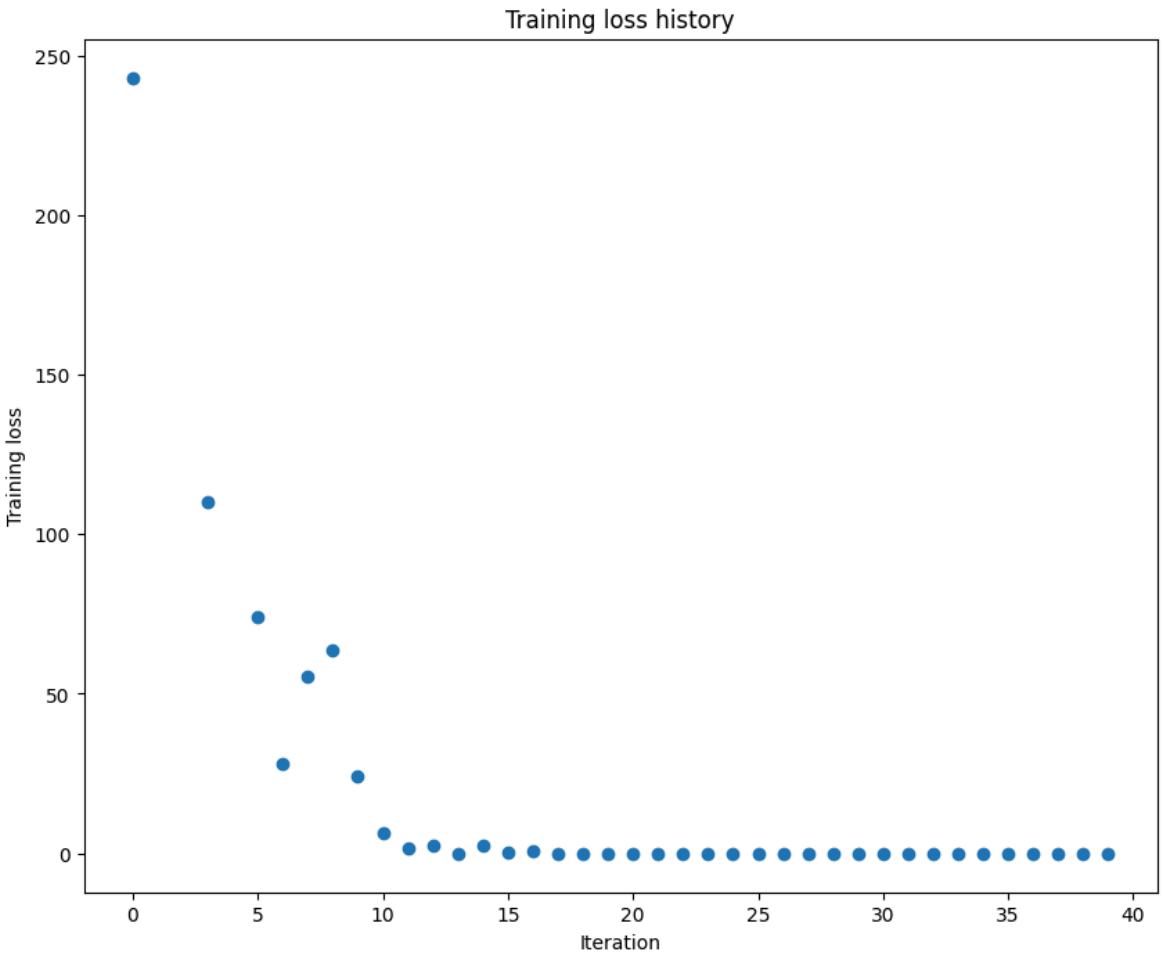
In [12]:
```
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
  print('Running check with reg = {}'.format(reg))
  model = FullyConnectedNet([H1, H2], input_dim=D, num_classes=C,
                            reg=reg, weight_scale=5e-2, dtype=np.float64)

  loss, grads = model.loss(X, y)
  print('Initial loss: {}'.format(loss))

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=Fal
    print('{} relative error: {}'.format(name, rel_error(grad_num, grads[
```

```
Running check with reg = 0
Initial loss: 2.3055605124584995
W1 relative error: 6.900614952955034e-07
W2 relative error: 7.067318330894913e-07
W3 relative error: 1.057907218310682e-06
b1 relative error: 8.905019556255785e-09
b2 relative error: 7.279112645062283e-09
b3 relative error: 1.1234512894697799e-10
Running check with reg = 3.14
Initial loss: 6.9959019240073195
W1 relative error: 2.254640657599829e-08
W2 relative error: 5.130950261133195e-08
W3 relative error: 1.3100007088632137e-08
b1 relative error: 5.710409607034686e-08
b2 relative error: 9.077066296928442e-09
b3 relative error: 1.786772496870058e-10
```

In [13]:
```
# Use the three layer neural network to overfit a small dataset.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}


#### !!!!!!
# Play around with the weight_scale and learning_rate so that you can ove
# Your training accuracy should be 1.0 to receive full credit on this par
# weight_scale = 1e-2
# learning_rate = 1e-4
weight_scale = 1e-1
learning_rate = 1e-3

model = FullyConnectedNet([100, 100],
              weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
                print_every=10, num_epochs=20, batch_size=25,
                update_rule='sgd',
                optim_config={
                  'learning_rate': learning_rate,
```

```
                }
        )
solver.train()

plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
```

```
(Iteration 1 / 40) loss: 243.006621
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.122000
(Epoch 1 / 20) train acc: 0.220000; val_acc: 0.137000
/Users/alanhsiu/Library/Mobile Documents/com~apple~CloudDocs/UCLA/Neural N
etworks and Deep Learning/HW/HW3/code/nndl/layers.py:194: RuntimeWarning:
divide by zero encountered in log
  loss = -np.sum(np.log(probs[np.arange(N), y])) / N
(Epoch 2 / 20) train acc: 0.520000; val_acc: 0.135000
(Epoch 3 / 20) train acc: 0.700000; val_acc: 0.148000
(Epoch 4 / 20) train acc: 0.760000; val_acc: 0.147000
(Epoch 5 / 20) train acc: 0.880000; val_acc: 0.143000
(Iteration 11 / 40) loss: 6.235880
(Epoch 6 / 20) train acc: 0.940000; val_acc: 0.142000
(Epoch 7 / 20) train acc: 0.960000; val_acc: 0.142000
(Epoch 8 / 20) train acc: 0.980000; val_acc: 0.143000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.143000
(Iteration 21 / 40) loss: 0.000000
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.143000
(Iteration 31 / 40) loss: 0.000000
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.143000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.143000
```


Training loss history
Training loss
Iteration
250
200
150
100
50
0
0
5
10
15
20
25
30
35
40